GOOD/BOOKS

Personalized Book Recommendation Engine

Project Documentation

Live Demo: https://goodbooks.pratham.dpdns.org
Documentation / Source Code: https://github.com/gititpratham/Goodbooks

A full-stack, ML-powered book recommendation app built on the Goodbooks-10K dataset, currently deployed on a live Oracle Cloud Infrastructure (OCI) instance.

Contents

1 Overview

GOOD/BOOKS is a personalized book recommendation web application. A user fills out a short preference form — genres, mood, popularity, minimum rating, book length, and era — and the system returns their top 10 matched books, ranked by a trained neural network.

The recommendation engine is a **hybrid machine-learning system** that combines:

- A **Multi-Layer Perceptron (MLP)** classifier trained on more than **424,000 pairwise user–book interactions**.
- **TF-IDF + SVD** description embeddings for semantic content understanding of each book's blurb.
- **Genre and mood multi-hot encoding** for explicit preference alignment.
- **Hard filters** for rating, page count, publication era, and popularity, applied on top of the ML ranking.

The project began as an exploratory data-analysis exercise on the Goodbooks-10K dataset and grew into a fully deployed, containerized, end-to-end product: data cleaning and enrichment, model training, a FastAPI backend, and a React/TypeScript frontend, all shipped together with Docker Compose and running live on an Oracle Cloud Infrastructure (OCI) instance.

2 Why This Project

2.1 Motivation

Book discovery is a problem almost every reader has felt firsthand — too many books, too little signal on which ones are actually worth the time. That made it a compelling, personally relatable domain to build a *complete* product around, rather than stopping at a notebook full of exploratory plots. Goodbooks was built to answer a simple question: given a raw, publicly available dataset, how far can it be taken — through cleaning, enrichment, modelling, and deployment — before it becomes something a real person could actually use to find their next book?

2.2 What Makes It Special

- **It is a genuine ML system, not a lookup table.** Recommendations come from a trained `MLPClassifier` scoring every book in the catalog against a learned user preference vector, not from static rules or simple genre matching.
- **Hybrid signal design.** The model blends structured signals (genre/mood multi-hot, numeric features) with unstructured signal (TF-IDF + SVD embeddings of book descriptions), so recommendations respond to both explicit taste and semantic content.
- **Data enrichment, not just data display.** The underlying Goodbooks-10K dataset was incomplete — many books were missing publication years, and several descriptions were thin or uninformative. Missing publication years were filled in, and richer, more useful descriptions were generated using a **locally-run LLM** rather than shipping the raw data as-is.
- **Cost-conscious, repeatable enrichment.** Using a local LLM for description generation avoided reliance on paid third-party APIs and external rate limits, keeping the enrichment pipeline free to run and easy to repeat as the dataset evolved.

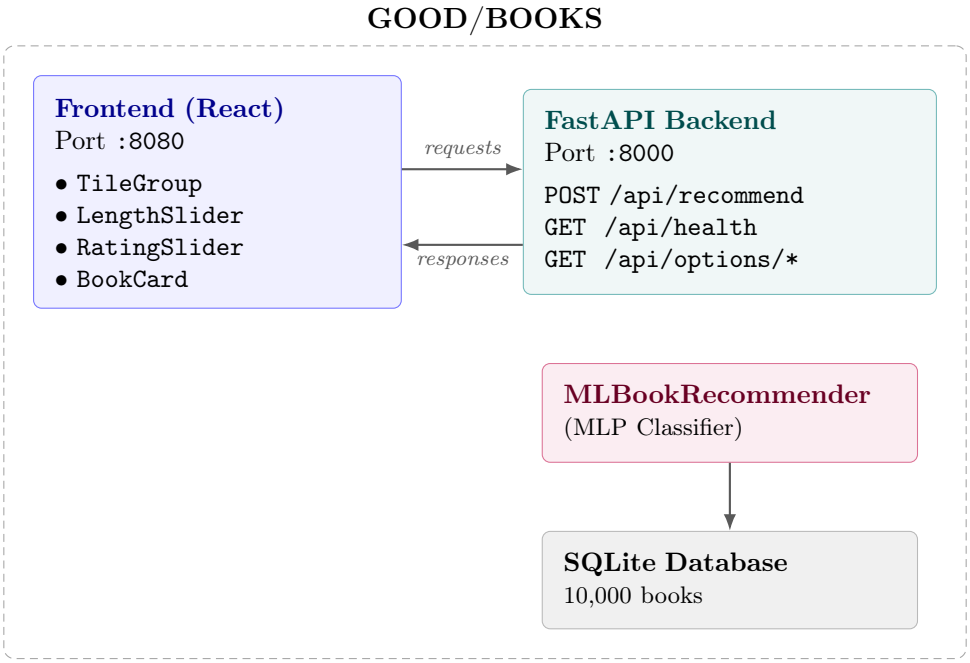
- **End-to-end ownership.** The project spans data cleaning, feature engineering, model training and evaluation, a documented REST API, a polished frontend, containerization, and live cloud deployment — a complete pipeline rather than an isolated script.
- **Real, hosted product.** Goodbooks isn't just source code sitting in a repository; it is deployed and publicly usable at `https://goodbooks.pratham.dpdns.org`, running on a live Oracle Cloud Infrastructure instance.

3 Tech Stack

Layer	Technology
Frontend	React 18 + TypeScript + Vite
Styling	Vanilla CSS (no framework)
Backend API	Python 3.11 + FastAPI + Uvicorn
ML Model	scikit-learn `MLPClassifier`
Feature Engineering	TF-IDF, TruncatedSVD, StandardScaler
Database	SQLite (WAL mode, seeded on first boot)
Containerization	Docker + Docker Compose
Dataset	Goodbooks-10K (Kaggle)

4 Architecture

Both services (frontend and backend) run inside Docker containers and communicate over an internal bridge network (`goodbooks-net`). The SQLite database is persisted via a named Docker volume (`goodbooks-data`).



On **first boot**, the backend automatically seeds the SQLite database from `backend/db/`. This takes roughly 10–15 seconds; the database then persists via the Docker volume, so subsequent starts are instant.

5 Project Structure (Part I)

```
goodbooks/
├── docker-compose.yml    # orchestrates frontend + backend containers
├── README.md
└── backend/    # FastAPI application + ML engine
    ├── Dockerfile
    ├── requirements.txt
    ├── main.py    # API entry point, route definitions
    ├── models.py    # Pydantic request/response models
    ├── database.py    # SQLite schema + seeding logic
    ├── ml_recommender.py    # MLBookRecommender inference class
    ├── db/
    │   ├── books_enriched.csv    # 10K books:  genres, moods, pages
    │   ├── tags.csv    # Goodreads tag vocabulary
    │   └── book_tags.csv    # book-tag association counts
    └── model/
        └── recommender.joblib    # active production model
```

6 Project Structure (Part II)

```
goodbooks/
├── frontend/    # React + TypeScript UI
│   ├── Dockerfile
│   ├── nginx.conf    # Nginx reverse-proxy config
│   ├── index.html
│   ├── package.json
│   ├── vite.config.ts
│   └── src/
│       ├── App.tsx    # root component + form state
│       ├── constants.ts    # genre/mood lists + filter defaults
│       ├── types.ts    # TypeScript interfaces (mirrors Pydantic)
│       ├── index.css
│       ├── api/
│       │   └── recommend.ts    # API client (fetch wrapper)
│       └── components/
│           ├── Header.tsx
│           ├── Hero.tsx
│           ├── TileGroup.tsx
│           ├── LengthSlider.tsx
│           ├── RatingSlider.tsx
│           └── BookCard.tsx
└── archive/    # development history, not used by the running app
    ├── train_model.py    # training script - run to retrain
    ├── recommender.py    # legacy SQL recommender (replaced by ML)
    ├── build_enriched.py    # builds books_enriched.csv from SQLite
    ├── goodbooks_eda.py    # exploratory data analysis
    ├── generate_descriptions.py    # description generation (local LLM)
    ├── archive_eda.py    # early EDA iteration
    └── eda/    # EDA output charts (PNG)
```

7 How the Recommender Works

7.1 ML Model Training

The model is a **pairwise preference MLP classifier** trained on the Goodbooks-10K ratings dataset. The training script lives at `archive/train_model.py`.

Training steps:

1. **Load data** — `books_enriched.csv` (10,000 books) and `ratings.csv` (981,756 ratings across 53,424 users).
2. **Engineer book features** — each book is represented as a **97-dimensional vector**:
   - Genre multi-hot (39 dims) — derived from the Goodreads tag mapping.
   - Mood multi-hot (8 dims) — inferred from description keywords.
   - Numeric features (4 dims) — pages, publication year, average rating, log(ratings_count).
   - TF-IDF + TruncatedSVD description embedding (46 dims).
3. **Build training pairs** — for each user, their rated books are split into:
   - **Positive** (liked): books rated ≥ 4 stars.
   - **Negative** (disliked): books rated ≤ 2 stars.

   Each training input concatenates the user preference vector with the book feature vector: concat(user_pref_vec[97], book_feat_vec[97]) = 194 dimensions.
4. **Train** — `sklearn.neural_network.MLPClassifier` with early stopping.
5. **Evaluate** — ROC-AUC and Precision/Recall computed on a held-out 15% test split.
6. **Save** — the serialized model bundle is written to `backend/model/recommender.joblib`.

Final model performance:

```
ROC-AUC = 0.8576
Accuracy = 77.1%
Precision: 0.73 | Recall: 0.78
Training pairs: 424,202 | Users: 53,424
```

7.2 Inference Pipeline

When a user submits the preference form, `POST /api/recommend` is called, and the following happens inside `MLBookRecommender.recommend()`:

1. **Build** a 97-dimensional user preference vector from the selected genres, moods, rating threshold, page limit, and era.
2. **Tile** the user vector against all 10,000 book feature vectors, forming $X = \text{concat}(\text{user}[97], \text{book}[97])$ with shape $(10000, 194)$.
3. **Score all books** in a single batch forward pass: scores = `mlp.predict_proba`$(X)[:, 1]$ — i.e. $P(\text{liked})$ for every book in the catalog.
4. **Apply hard filters**, dropping any book that fails:
   - `average_rating < minRating`
   - `pages > maxPages` (when a limit is set)

- `pub_year` outside the selected era range
- `ratings_count > 65,000` when `popularity == "underrated"`

5. **Sort** the remaining books by ML score, descending.
6. **Return** the top 10 results.

7.3 Filters Explained

Filter	Field	Behaviour
Genres	`genres[]`	Influences the genre multi-hot component of the preference vector.
Mood	`moods[]`	Steers the MLP score via the mood multi-hot encoding.
Popularity	`popularity`	`popular` = no filter; `underrated` = excludes books with $> 65{,}000$ ratings (roughly the top 15%).
Min Rating	`minRating`	Hard filter — books below this average rating are excluded.
Max Runtime	`maxPages`	Hard filter on page count. Conversion: 50 pages/hour (e.g. 30 hrs $\Rightarrow$ 1,500 pages).
Era	`pubEra`	`recent` = year ≥ 2000, `classic` = year < 1980, `any` = no filter.

8 API Reference

Base URL (local): `http://localhost:8000`
Swagger docs: `http://localhost:8000/docs`

`POST /api/recommend`

Returns up to 10 personalized book recommendations.

Request body:

```
{
  "genres": ["Fantasy", "Thriller"],
  "moods": ["Cozy", "Escapist"],
  "minRating": 4.0,
  "maxPages": 1500,
  "pubEra": "recent",
  "popularity": "popular"
}
```

Response:

```
{
  "books": [
    {
      "title": "The Name of the Wind",
      "author": "Patrick Rothfuss",
      "genres": ["Fantasy", "Fiction"],
      "moods": [],
      "pitch": "A legendary figure recounts his life story...",
```

```
      "match": 92,
      "average_rating": 4.55,
      "ratings_count": 497765,
      "pages": 662,
      "image_url": "https://...",
      "pub_year": 2007
    }
  ],
  "count": 10,
  "query": { ... }
}
```

`GET /api/health`

Returns backend health status and whether the database has been seeded.

`GET /api/options/genres`

Returns the available genre labels, sourced from the loaded ML model.

`GET /api/options/moods`

Returns the available mood labels, sourced from the loaded ML model.

9 Install

9.1 Prerequisites

- Docker Desktop installed and running (recommended path), *or*
- Python 3.11 and Node.js, if running the backend and frontend without Docker.

9.2 Quick Start with Docker

```
# 1. Clone the repository
git clone https://github.com/gititpratham/Goodbooks.git
cd Goodbooks

# 2. Build and start all services
docker compose up --build

# if the above command fails to build the image, run:
sudo docker compose up --build

# Frontend -> http://localhost:8080
# API Docs -> http://localhost:8000/docs
```

On first boot, the backend automatically seeds the SQLite database from `backend/db/`, which takes about 10–15 seconds. The database persists via a Docker volume, so subsequent starts are instant.

Rebuild a single service:

```
docker compose build --no-cache frontend
docker compose up -d frontend
```

Stop all containers:

```
docker compose down
```

9.3 Running without Docker

Backend:

```
cd backend
pip install -r requirements.txt
uvicorn main:app --reload --host 0.0.0.0 --port 8000
```

Frontend:

```
cd frontend
npm install
npm run dev
# Dev server starts at http://localhost:5173
```

10 Usage

10.1 Using the Hosted Version

No installation is required to try the project — it is publicly hosted at:

https://goodbooks.pratham.dpdns.org

10.2 Using a Local Instance

Once the containers are running (or the dev servers are started), open:

- Frontend: `http://localhost:8080` (Docker) or `http://localhost:5173` (dev server)
- API docs: `http://localhost:8000/docs`

10.3 Typical Workflow

1. Select preferred genres and moods using the tile filters.
2. Choose a popularity preference (`popular` or `underrated`).
3. Set a minimum star rating and a maximum book runtime.
4. Select an era (recent, classic, or any).
5. Submit the form — the app calls `POST /api/recommend` and displays the top 10 matched books as cards, each with a match score, cover, author, genres, and description.

10.4 Filter Defaults

Pre-selected when the page loads:

Filter	Default
Genres	Fantasy, Sci-Fi, Thriller
Mood	Cozy
Popularity	Popular
Min Rating	★ 4.0
Max Book Runtime	30 hrs ($\leq$ 1,500 pages)
Era	Recent ($\geq$ 2000)

10.5 Retraining the Model

If `books_enriched.csv` is updated, or the model needs to be retrained from scratch:

```
python archive/train_model.py
```

This will:

1. Load `backend/db/books_enriched.csv` and `backend/db/ratings.csv`.
2. Build training pairs from all 53K users.
3. Train and evaluate a new MLP classifier.
4. Save the result to `backend/model/recommender.joblib`.

Then rebuild the backend to pick up the new model:

```
docker compose build --no-cache backend && docker compose up -d backend
```

11 Technical Documentation

11.1 Frontend Overview

Component	Purpose
`Header`	App logo and navigation bar
`Hero`	Landing headline and tagline
`TileGroup`	Multi-select tiles for Genres, Moods, and Era
`LengthSlider`	Max Book Runtime slider (10 / 15 / 20 / 30 / 40 hrs / No Limit)
`RatingSlider`	Minimum Star Rating slider (0.0 – 5.0)
`BookCard`	Book cover, title, author, description, genre chips, rating, page count, year

11.2 Backend Overview

- `main.py` — FastAPI entry point and route definitions (`/api/recommend`, `/api/health`, `/api/options/*`).
- `models.py` — Pydantic request/response models, mirrored on the frontend by `types.ts`.
- `database.py` — SQLite schema definition and first-boot seeding logic.
- `ml_recommender.py` — the `MLBookRecommender` class, which loads `recommender.joblib` and performs batch inference over the full catalog.

11.3 Key Design Decisions

- **Batch scoring over the whole catalog** at request time (10,000 books scored in a single forward pass) keeps inference simple and fast enough for interactive use, without needing an approximate nearest-neighbour index.
- **Hard filters applied after ML scoring** keep the neural network focused purely on preference ranking, while deterministic constraints (rating, pages, era, popularity) are enforced separately and predictably.
- **Local LLM for description enrichment**, run as an offline preprocessing step rather than at request time, to avoid API costs/rate limits and keep the live application fast and predictable.
- **SQLite with first-boot seeding** keeps deployment simple — no external database service is required, and the container is self-contained after the first startup.
- **Docker Compose with two isolated services** (frontend served via Nginx, backend via Uvicorn/FastAPI) communicating over an internal bridge network, mirroring how the app is deployed in production on Oracle Cloud Infrastructure.

12 Data Preparation & Tools

12.1 Dataset

The project is built on **Goodbooks-10K** by Zygmunt Zając (Kaggle):

- 10,000 books (the most popular books on Goodreads).
- 981,756 user ratings from 53,424 users.
- Tag/genre metadata sourced from Goodreads bookshelves.

Files under `backend/db/`:

File	Description
`books_enriched.csv`	10K books with genre, mood tags, and page counts
`tags.csv`	∼34K unique Goodreads tag names
`book_tags.csv`	∼1M book–tag count associations

12.2 Preparation Steps

1. **Exploratory Data Analysis** (`archive/goodbooks_eda.py`, `archive_eda.py`) — the raw dataset was inspected to understand rating distributions, tag frequency, missing fields, and overall data quality; output charts were saved to `archive/eda/`.
2. **Fill missing publishing years** — a meaningful share of entries in the original dataset lacked a publication year. These were identified and backfilled so the dataset could support reliable era-based filtering (recent vs. classic) in the final application.
3. **Generate richer descriptions** — entries with missing or thin descriptions were enriched using a **locally-run LLM** (`archive/generate_descriptions.py`), producing more informative, human-readable summaries for each book without relying on a paid external API.
4. **Build the enriched dataset** (`archive/build_enriched.py`) — genre and mood tags, page counts, and the enriched descriptions/publication years were merged from the raw SQLite dump into a single working file, `books_enriched.csv`.

5. **Feature engineering for the model** — each of the 10,000 enriched books was converted into a 97-dimensional feature vector (genre multi-hot, mood multi-hot, numeric features, and TF-IDF + SVD description embeddings), as described in Section 4 (*How the Recommender Works*).
6. **Validate and export** — the cleaned, enriched dataset was validated and loaded into the SQLite database that ships with the backend and is seeded automatically on first boot.

12.3 Tools Used

- **Goodbooks-10K dataset (Kaggle)** — source data for books, ratings, and tags.
- **Local LLM** — used offline to generate improved book descriptions, avoiding paid APIs and external rate limits.
- **Python data-processing stack** (pandas-style scripting) — used for cleaning, transforming, and merging enriched fields back into the working dataset.
- **scikit-learn** — `TfidfVectorizer`, `TruncatedSVD`, `StandardScaler`, and `MLPClassifier` for feature engineering, dimensionality reduction, and model training.
- **SQLite** — the storage format for the enriched dataset, seeded into the running application on first boot.

13 Archive Folder

The `archive/` directory holds development scripts kept for reference; these are **not** used by the running application, but document how the project's data and model were built.

File	Purpose
`train_model.py`	ML training pipeline — run to retrain the model
`recommender.py`	Legacy SQL heuristic recommender (replaced by ML)
`build_enriched.py`	Built `books_enriched.csv` from the raw SQLite dump
`goodbooks_eda.py`	Exploratory Data Analysis of the raw dataset
`generate_descriptions.py`	Utility to generate book description summaries (local LLM)
`eda/`	EDA output charts (PNG images)

Links

- **Live Demo:** https://goodbooks.pratham.dpdns.org
- **Docs / Source Code:** https://github.com/gititpratham/Goodbooks
- **Dataset:** Goodbooks-10K on Kaggle